

MUGEN: A Unified Framework for Efficient Motion Understanding and Generation

Zhankai Ye¹ Yukai Jin¹ Bingyang Wei² Bofan Li¹ Yusen Wu³
Fangyi Li⁴ Shangqian Gao¹ Xin Liu¹

¹Florida State University ²Texas Christian University ³University of Miami ⁴University of Pennsylvania

 <https://github.com/JYe16/MUGEN>

 <https://huggingface.co/zy22b/MUGEN>

Abstract

Grounding human motion in language, and language in motion, is a central step toward physical AI systems that can understand, generate, and communicate human behavior. Unified motion–language systems first coupled the two directions through a shared discrete motion codebook, but quantization limits generation quality. The strongest generators buy quality back at growing cost: stacked residual codebooks enlarge the representation; masked decoding stages, long autoregressive rollouts, and denoising chains of tens to hundreds of steps stretch inference; even the continuous-latent designs among them reach their latent only through an iterative diffusion head; and none of this decoding machinery serves understanding. We therefore propose MUGEN, a unified motion–language framework that pays neither cost: *no codebook, one draw*. A single adaptive-length autoencoder compresses any-length motion into a few continuous latent slots, the system’s only motion representation: the language model generates them for text-to-motion and reads them back for motion understanding. Depth-routed hidden states let each slot read from the transformer depth it needs, and a calibrated head predicts a joint distribution over the full latent set, so a single draw carries the text-conditional, cross-slot variation a description permits. At a decoding cost of K language-model steps, one draw, and one decoder pass, MUGEN leads language-model baselines on FID on HumanML3D while raising retrieval precision above the real-motion reference under the standard evaluator, achieves the best CIDEr and BLEU@4 scores, and surpasses the discrete-token state of the art on every retrieval and alignment metric on SnapMoGen.

served motions in human language. Since these two tasks describe the same underlying behavior, it is desirable to handle them within a unified framework built around a shared motion representation. Existing systems such as TM2T and MotionGPT take an important step in this direction by representing motion through a VQ-VAE-style discrete interface and training a shared architecture for both text-to-motion generation and motion-to-text understanding (Guo et al., 2022b; Jiang et al., 2023). However, the quantized codebook limits reconstruction and generation quality. More recent generators recover quality only at added cost: residual codebook hierarchies with staged masked prediction enlarge the motion representation, autoregressive motion-token decoding and diffusion-based denoising stretch inference to tens or hundreds of model passes, and even the continuous-latent designs among them must sample through an iterative diffusion head (Guo et al., 2024, 2025; Zhang et al., 2023a, 2024; Zhu et al., 2025). None of this added machinery serves the understanding branch. As a result, current systems can unify the language backbone, but high-quality and efficient motion generation and motion understanding still do not operate through the same motion interface.

This paper asks whether one continuous motion representation can support both generation and understanding without paying either cost. Our insight is that motion does not need a discrete codebook to enter a language model. A motion clip can instead become a small set of continuous latent slots that represents the sequence, gives the language model a compact generation target, and serves as the language model’s input for motion understanding.

We present MUGEN, a unified motion–language framework built around that interface under the opposite budget: *no codebook, one draw*. The budget itself is not new: early sequence-level VAEs such as TEMOS already decode a whole motion from one Gaussian draw (Petrovich et al., 2022), and

1 Introduction

Human motion provides a natural interface between language and embodied behavior. A general motion–language system should therefore support both directions of this interaction: generating plausible motions from textual descriptions and explaining ob-

the generators above outperform them at growing decoding cost. What MUGEN claims is the machinery that makes a single draw competitive again, inside a language model and on a representation that understanding shares. Because the slots are continuous and learned jointly with the decoder rather than selected from a fixed vocabulary, the shared interface is no longer capped by codebook capacity, so unifying the two directions no longer sacrifices generation quality: joint training improves FID over a generation-only twin (Sec. 4.5). A single Adaptive-Length AutoEncoder (ALAE) maps any-length motion into K continuous latent slots and decodes those slots back to frames. For generation, the language model runs a K -step rollout seeded by a special <MOT> token, predicts the whole latent set, and sends one sampled latent set to the frozen decoder. For understanding, the frozen encoder extracts the same K slots from an observed motion, and a projector maps them into the language-model embedding space for caption generation. Both directions share the language model, the motion interface, and the continuous representation.

MUGEN makes this compact interface work with two design choices. First, because the latent set is small, each slot must carry a large and distinct part of the sequence; if every slot reads the final layer, they all receive the same kind of evidence and can differ only by position. Depth-routed hidden states remove this constraint by letting each slot read from the transformer depth it needs, with the final-layer interface retained as the special case where every slot routes to the last layer. Second, a single draw must supply all the variation a description permits, and independent per-dimension noise cannot express variation shared across slots. A calibrated low-rank factor head therefore predicts a joint distribution over the full latent set, so one draw carries text-conditional variation that spans slots. Together, these components replace codebook stages, masked refinement, and denoising chains with K language-model rollout steps, one draw, and one decoder pass.

We evaluate MUGEN on HumanML3D and SnapMoGen under a fixed protocol. The same sampler supplies all reported metrics, and test results follow the official protocols. On HumanML3D, MUGEN leads language-model baselines on FID, retrieval precision, and matching distance, and achieves the best CIDEr and BLEU@4 captioning scores, while masked-codebook pipelines keep the best FID. On SnapMoGen, it further improves over the discrete-

token state of the art on every retrieval rank and on CLIP alignment.

Our key contributions are as follows:

- We show that high-quality motion generation and motion understanding can share one compact continuous motion representation, without the codebook hierarchies and iterative decoding that current generators pay for quality.
- MUGEN realizes this interface with ALAE latent slots, depth-routed hidden states that draw slot-specific evidence from the full depth of the language model, and a calibrated low-rank latent head whose single draw carries text-conditional variation across slots.
- MUGEN leads strong motion-language baselines on every retrieval rank, on generation FID and matching distance, and on BLEU@4 and CIDEr, while decoding each motion in K language-model steps and one draw: 9 ms per motion, an order of magnitude less inference compute, and 6–14 $\times$ lower latency than the strongest masked-codebook and unified baselines in a head-to-head measurement.

2 Related Work

Motion Generation. Text-to-motion generation is measured on paired benchmarks such as HumanML3D (Guo et al., 2022a) and the expressive whole-body Motion-X (Lin et al., 2023). Early continuous-latent models learn a conditional motion distribution or align a motion autoencoder with CLIP space (Tevet et al., 2022a; Radford et al., 2021); TEMOS, for instance, decodes a whole motion from a single draw of a sequence-level VAE (Petrovich et al., 2022). Token-based systems cast motion as a discrete sequence: TM2T models the two modalities reciprocally (Guo et al., 2022b), T2M-GPT predicts motion tokens autoregressively (Zhang et al., 2023a), AttT2M adds body-part and global-local text attention over motion tokens (Zhong et al., 2023), and MotionGPT shares a language-model vocabulary (Jiang et al., 2023), at a cost that scales with token count; recent work scales tokenizer, model, and data together (Lu et al., 2025; Fan et al., 2025). Diffusion models such as MDM and MotionDiffuse instead denoise continuous trajectories (Ho et al., 2020; Tevet et al., 2022b; Zhang et al., 2024), optionally in a compressed latent space (Rombach et al., 2022), with retrieval-augmented (Zhang et al., 2023b), physics-guided (Yuan et al., 2023), or consistency-distilled (Song et al., 2023; Dai et al., 2024) variants. Masked generative models refine tokens itera-

tively (Chang et al., 2022; Pinyoanuntapong et al., 2024b,a); MARDM transfers masked autoregression to continuous latents sampled through a diffusion head (Meng et al., 2025).

These generators rest on a motion representation. Discrete designs follow vector quantization (Van Den Oord et al., 2017), from single vocabularies to the residual codebooks and staged decoding of MoMask (Guo et al., 2024; Lee et al., 2022) and MoMask++ (Guo et al., 2025). Our sampling budget of one draw and one decoder pass predates all of these: TEMOS already paid it, though at a quality every generator above has since surpassed. The closest generation-only continuous design is MLD, which likewise compresses a clip into a few latent tokens (Chen et al., 2023); that latent exists to host a diffusion process, so it is reachable only by iterative denoising and never leaves the generator. MUGEN lets the language model write and read the slots directly, and replaces the denoising chain with one draw from a predicted joint distribution (Sohn et al., 2015). Its claim is not the budget but what makes it competitive: behind K cached language-model steps, depth-routed readout and a calibrated low-rank covariance bring single-draw continuous generation into the range of multi-stage discrete pipelines—ahead on retrieval and alignment, behind the masked-codebook family on FID—using the same slots read by the understanding branch.

Motion Understanding. TM2T first coupled captioning and generation in one tokenized framework; MotionGPT trained a shared language model across motion–language tasks, and MotionGPT3 replaced discrete motion symbols with continuous features (Zhu et al., 2025). UniMo couples the two directions through chain-of-thought supervision and reinforcement post-training (Wang et al., 2026). TMR learns a contrastive text–motion space for retrieval (Petrovich et al., 2023), and MG-MotionLLM addresses comprehension and generation at multiple granularities (Wu et al., 2025). GeoMotionGPT aligns the motion codebook with the language-model embedding space through orthogonal sparse projection (Ye et al., 2026). These reciprocal systems couple the two directions either through a shared discrete token interface, whose quantization limits generation quality, or, in MotionGPT3, through continuous latents reachable only by an iterative diffusion head. MUGEN keeps the shared continuous interface but reaches it in one calibrated draw: one autoencoder yields latent slots

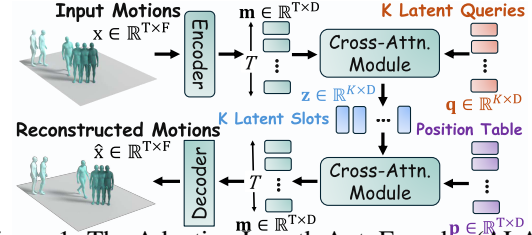


Figure 1: The Adaptive-Length AutoEncoder (ALAE). The encoder compresses a motion of arbitrary length into exactly K continuous latent slots by letting learned latent queries cross-attend to a temporal memory. The decoder reconstructs the motion directly from those slots.

serving as generation target, understanding input, and decoder input.

3 Methodology

3.1 Adaptive-Length AutoEncoder Without a Codebook

Removing the codebook leads to two requirements. First, the representation must map variable-length motion into a fixed budget. Second, that budget must preserve enough structure for direct reconstruction.

We therefore build a continuous latent interface around a compact autoencoding problem. Given a normalized motion sequence $\mathbf{x} = [\mathbf{x}_1, \dots, \mathbf{x}_T] \in \mathbb{R}^{T \times F}$, where F is the frame-feature dimension ($F=263$ for HumanML3D and $F=296$ for SnapMoGen), the autoencoder reconstructs $\hat{\mathbf{x}} \in \mathbb{R}^{T \times F}$ through exactly K continuous slots, $\mathbf{z} = [\mathbf{z}_1, \dots, \mathbf{z}_K] \in \mathbb{R}^{K \times D}$ with $D=512$. The latent budget K is the only interface size exposed to later modules.

The first step is to turn the input sequence into a temporal memory that keeps local motion structure accessible. ALAE (Fig. 1) applies a one-dimensional convolutional backbone with dilated ResNet blocks to produce $\mathbf{m}_{1:T} \in \mathbb{R}^{T \times D}$, which preserves framewise temporal structure while normalizing the feature space for the latent bottleneck.

The second step is to compress this variable-length memory into a fixed set of continuous latent slots. Instead of assigning frames to discrete entries, learned latent queries perform compression directly, following the fixed latent-bottleneck principle of cross-attention architectures (Jaegle et al., 2021). The construction is also related to attention-based set encoders (Lee et al., 2019) and learned slot representations (Locatello et al., 2020); in the image domain, 1D tokenizers likewise compress inputs of arbitrary resolution into a small, controllable token budget (Patel et al., 2026). Let $\mathbf{Q} = [\mathbf{q}_1, \dots, \mathbf{q}_K] \in \mathbb{R}^{K \times D}$ be the trainable latent queries. A stack of cross-attention blocks updates these queries by

attending to the temporal memory:

$$\mathbf{z} = E_\phi(\mathbf{x}) = \text{LN}(\text{CA}_\phi(\mathbf{Q}, \mathbf{m}_{1:T})).$$

Each block contains self-attention among the latent queries, cross-attention to the temporal memory, and a feed-forward network. The resulting slots summarize complementary aspects of the motion rather than fixed temporal windows. To stabilize this specialization, an orthogonality penalty discourages off-diagonal cosine similarity between query vectors:

$$\mathcal{L}_{\text{orth}} = \frac{1}{K(K-1)} \sum_{i \neq j} \left(\frac{\mathbf{q}_i^\top \mathbf{q}_j}{\|\mathbf{q}_i\|_2 \|\mathbf{q}_j\|_2} \right)^2.$$

For $K=1$ the pair sum is empty and we define $\mathcal{L}_{\text{orth}}=0$.

The third step is to decode directly from the same slots. The decoder samples T sinusoidal phase queries from a shared position table, so frame t is conditioned on its relative phase in the sequence rather than a fixed absolute index. These queries attend to $\mathbf{z}_{1:K}$ through cross-attention blocks and are refined by a temporal convolutional decoder:

$$\hat{\mathbf{x}} = D_\psi(\mathbf{z}, T) = R_\psi(\text{CA}_\psi(\mathbf{p}_{1:T}, \mathbf{z}_{1:K})).$$

Because K is fixed while T is supplied explicitly, the autoencoder can train on variable-length clips without changing the interface size seen by later modules.

Training encourages the slots to remain reconstructive rather than merely descriptive. We use masked Smooth-L1 reconstruction over all frame features, a joint-position Smooth-L1 term, a perceptual loss from a frozen dataset-specific motion encoder, and the orthogonality penalty $\mathcal{L}_{\text{orth}}$.

After training, this autoencoder becomes the frozen motion interface. The checkpoint stores per-dimension latent mean and standard-deviation statistics; the generator standardizes all latent targets with these statistics while keeping the encoder and decoder fixed. From this point on, the K continuous slots are the representation used by both generation and understanding.

3.2 Latent Slot Generation with Depth Routing

Language-model motion generators read from the final transformer layer, an interface inherited from next-token prediction rather than designed for motion (Zhang et al., 2023a; Jiang et al., 2023), even though useful information is distributed across depth (Peters et al., 2018; Rogers et al., 2020). The mismatch sharpens in our setting: at the small budgets we target (Sec. 4.5), each of the K slots carries

a large fraction of the sequence, and slots that all read the final layer are forced to draw on the same kind of evidence, differing only by position. We therefore let each slot learn its own distribution over the L layers; reading only the final layer remains a special case, so depth routing strictly generalizes the conventional interface.

Initializing continuous latent slots via autoregressive rollout. The generator must produce K continuous latent slots that the frozen ALAE decoder can turn into frames. We therefore use the language model to create these slots (Fig. 2).

Given a text description y , the prompt ends with a special <MOT> token whose hidden state seeds a K -step rollout. At step $k=1$ the model takes this prompt state; at later steps it feeds the previous final-layer hidden state, after a LayerNorm feedback transform, back to the language model as a continuous input embedding. At each step, MUGEN retains the output of every transformer layer, producing a depth-indexed state tensor $\mathbf{H} \in \mathbb{R}^{K \times L \times H}$, where L is the number of transformer layers and H is the hidden width. These states form the evidence for latent prediction. Which layers each latent slot reads from is decided by the router described next.

Routing latent slots over depth. Routing is conditioned on the prompt. Latent queries imported from the ALAE encoder cross-attend to the projected prompt states and yield one context vector $\mathbf{c}_k \in \mathbb{R}^D$ per slot. The router logits are then formed as a *bounded residual* rather than a direct regression,

$$\mathbf{e}_k = s_s \tanh(\mathbf{g}_k / s_s) + s_d \tanh(\Delta_k(y) / s_d),$$

where the free table $\mathbf{g} \in \mathbb{R}^{K \times L}$ carries the prompt-independent part of the decision and $\Delta_k(y) = \text{MLP}(\mathbf{c}_k) \in \mathbb{R}^L$ is a per-prompt correction. Squashing the two branches separately bounds the total logit margin by $s_s + s_d$. This matters because an unconstrained router has a cheap way to suppress the sampling noise injected during training: grow the margins until the softmax saturates. Its Jacobian then vanishes, the prompt branch stops receiving gradient, and routing freezes on a prompt-independent pick it cannot escape. The split also removes any reason to spend the prompt branch on a constant, since $\mathbf{g}$ already represents one at no cost to adaptivity. We zero-initialize the correction head, so training starts at the static solution and the static router acts as a performance floor rather than a competing optimum.

Routing weights are Gumbel-Softmax sam-

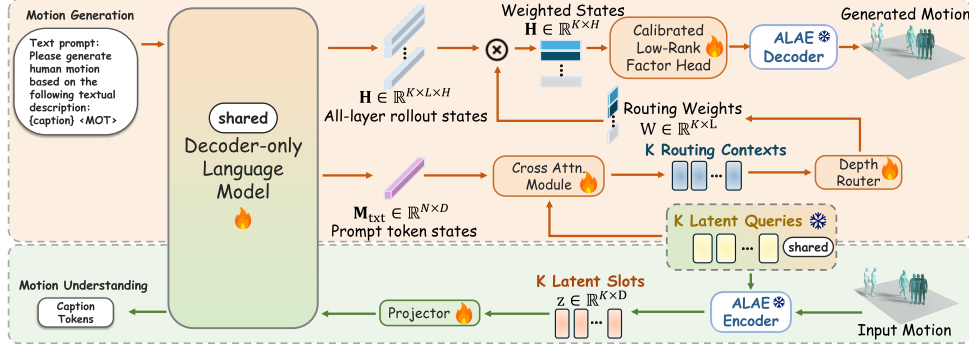


Figure 2: The shared motion-language interface of MUGEN; flames mark trainable modules and snowflakes frozen ones. **Generation (top):** a <MOT>-seeded K -step rollout keeps all L layer states $\mathbf{H}$, a prompt-conditioned router assigns each latent slot its own depth mixture $\mathbf{W}$, and the calibrated low-rank factor head draws the complete latent set in a single shot for the frozen ALAE decoder. **Understanding (bottom):** the frozen ALAE encoder maps an input motion to the same K slots, which a projector feeds to the same language model as caption context.

ples (Jang et al., 2017) during training and a deterministic tempered softmax $\mathbf{w}_k = \text{softmax}(\mathbf{e}_k / \tau_T)$ at evaluation, which keeps all reported metrics reproducible. The state handed to the head is the routed combination

$$\tilde{\mathbf{h}}_k = \sum_{l=1}^L w_{k,l} \mathbf{H}_{k,l}, \quad k = 1, \dots, K.$$

A bounded router is still free to ignore the prompt, so we add a small mutual-information term that rewards routing decisively within a prompt and differently across prompts. Because the logits depend only on the prompt, they are computed once per generation.

Producing latent slots with a calibrated low-rank factor head. The current architecture gives the model a single draw, so that draw must capture the variation a text description permits. A deterministic head therefore produces only one motion per text description, while independent per-dimension noise cannot express structured variation shared across latent slots. We therefore predict a structured latent distribution, implemented with a low-rank factor head over the routed slot states.

The factor head reads the routed slot states: each routed slot state $\tilde{\mathbf{h}}_k$ is layer-normalized and linearly projected to the per-slot mean and diagonal scale, and the slot-pooled normalized states produce the factor amplitudes. It models the flattened latent $\bar{\mathbf{z}} \in \mathbb{R}^{KD}$ with a low-rank-plus-diagonal Gaussian

$$q(\bar{\mathbf{z}} | y) = \mathcal{N}\left(\boldsymbol{\mu}(y), \mathbf{U} \text{diag}(\mathbf{a}(y))^2 \mathbf{U}^\top + \text{diag}(\boldsymbol{\sigma}(y)^2)\right),$$

where $\mathbf{U} \in \mathbb{R}^{KD \times r}$ is a learned global factor basis and $\mathbf{a}(y) = \exp(\ell(y)) \in \mathbb{R}^r$ is a text-conditioned amplitude vector, whose log-amplitudes are produced from the slot-pooled hidden states by a linear

layer. Because the basis spans *across* slots, a single draw carries both within-slot and cross-slot correlation. As $\mathbf{a} \rightarrow 0$ the family degenerates to the diagonal Gaussian, so the factor head strictly generalizes the diagonal model. Fitting its likelihood without stalling the mean requires the decoupled anchor of Sec. 3.3.

At inference a latent is drawn once, at a temperature τ that scales the zero-mean perturbation around $\boldsymbol{\mu}$; the draw is mapped back through the stored latent statistics, and the frozen ALAE decoder turns it into T frames. Setting $\tau=0$ yields the deterministic μ -decode variant used as a reference protocol in our experiments.

3.3 Joint Generation and Understanding

The same continuous latent-slot interface serves both tasks. Generation predicts latent slots from text, while motion understanding reads the same slots from the frozen encoder and maps them into the shared language model. No direction-specific tokenizer or decoder exists anywhere in the system.

Motion understanding branch. The frozen ALAE encoder converts an input motion into its K latent slots, a learned projector maps them into the language-model embedding space, and the decoder-only language model is trained with teacher-forced caption cross-entropy loss. Label smoothing, a train-time Gaussian perturbation of the input latents, and a flooding floor on the caption loss keep this branch from overpowering the shared backbone late in training. Both directions are optimized on the same batches and share the language backbone, the <MOT> interface, and the single motion representation.

Joint training objective. During training, the motion branch decodes the predicted mean, $\hat{\mathbf{x}} = D_\psi(\boldsymbol{\mu}, T)$, and applies the same masked recon-

struction, joint-position, and perceptual terms used to train ALAE. The full loss adds the decoupled anchor, the motion-to-text (M2T) understanding term, and the router’s mutual-information regularizer (Sec. 3.2):

$$\mathcal{L} = \mathcal{L}_{\text{motion}}(\mathbf{x}, \hat{\mathbf{x}}) + \lambda_{\text{lat}} \mathcal{L}_{\text{lat}} \\ + \lambda_{\text{m2t}} \mathcal{L}_{\text{m2t}} + \lambda_{\text{MI}} \mathcal{L}_{\text{MI}}.$$

The decoupled anchor $\mathcal{L}_{\text{lat}}$ is a Gaussian negative log-likelihood on the target latents $\mathbf{z}^*$, split into two terms with complementary stop-gradients: one trains the mean under a detached diagonal precision, the other fits the full low-rank covariance to the detached residual by exact maximum likelihood. Without that split, joint likelihood can explain mean error as inflated variance, weakening the mean update and stalling the motion prediction path. Because the covariance parameters never enter the mean term, the mean gradient stays stable while the amplitude $\mathbf{a}(y)$ self-calibrates without clamps or schedules. No KL term is needed: likelihood alone calibrates the predicted scale, and the learned $\sigma(y)$ becomes genuinely text-dependent.

The ALAE encoder and decoder stay frozen throughout, and the router, the factor head, the feedback transform, the understanding projector, and the language backbone are optimized jointly by this single objective.

4 Experiments

4.1 Setup

We evaluate on two text-to-motion benchmarks that differ in scale, caption style, and motion representation: **HumanML3D** (263-d frame features at 20 fps, short crowd-sourced captions) and **SnapMoGen** (longer, more expressive captions), each with its own official splits and evaluator. Generation is scored with FID (Heusel et al., 2017), R-Precision (R@1/2/3) (Aslam and Yilmaz, 2005), matching distance (MM-Dist), Diversity, and Multi-Modality (MM), plus SnapMoGen’s CLIP score; understanding is scored with BLEU (Papineni et al., 2002), ROUGE, CIDEr (Vedantam et al., 2015), BERTScore (Zhang et al., 2020), and motion–text retrieval. All test numbers follow each benchmark’s official 20-replication protocol; we report means, with 95% confidence intervals where space permits.

Stage one trains one ALAE per latent budget K ($D=512$, four encoder and four decoder layers). Stage two freezes it and trains GPT-2 (Radford et al., 2019) as the language backbone ($L=12$ layers) with the depth router of Sec. 3.2 and the decoupled

anchor of Sec. 3.3 at factor rank $r=64$, learning generation and understanding jointly on the same batches. Sampling temperature and checkpoints are selected on the validation split alone; the appendix gives the full recipe and defines every metric.

The recipe is identical on both benchmarks and only the latent budget differs ($K=2$ on HumanML3D, $K=4$ on SnapMoGen); on HumanML3D a single checkpoint produces both the generation and the understanding numbers. Swapping GPT-2 for Qwen3 (Yang et al., 2025) or SmoLLM2 (Ben Allal et al., 2025) with LoRA adapters (Hu et al., 2022) never surpasses the flagship’s joint generation–understanding score at up to $14\times$ the parameters, so we keep the smallest backbone (ablation in the appendix).

4.2 Motion Generation

Tables 1 and 2 report generation, where the strongest baseline on each is a multi-stage discrete codebook pipeline. MUGEN has the best text–motion correspondence in both tables, on HumanML3D beyond the real-motion reference and on SnapMoGen ahead of both MoMask++ variants at every rank with CIs separated; six of the ten HumanML3D baselines also cross that reference by smaller margins, so part of the effect belongs to the evaluator. FID is the one metric where the discrete pipelines stay ahead. An oracle decode locates the cause in the conditional sampler rather than the representation: adding half of the true latent residual cuts SnapMoGen FID to 6.89, while batch-shuffled residuals collapse R@1 to 0.42, so the missing variance is text-conditional and directionally structured, and the gap is the price of the single draw, not of removing the codebook. Every column comes from one calibrated sampler at the validation-selected temperature, which favors fidelity; Diversity tracks the real-motion reference on both benchmarks, so the low MultiModality reflects the operating point rather than a collapsed distribution.

4.3 Motion Understanding

Table 3 evaluates the understanding branch, which reads the same frozen latent slots the generation branch predicts. MUGEN leads every reported retrieval rank, above the real-motion reference, together with BLEU@4 and CIDEr, at no captioning-specific training cost. UniMo, trained on curated chain-of-thought annotations, leads the remaining caption metrics but reports no motion-side retrieval, and matching distance is the one column where a

Type	Method	R@1↑	R@2↑	R@3↑	FID↓	MM-Dist↓	Diversity	MM
	Real motions	0.510	0.700	0.795	–	3.012	9.503	–
Gen. only	MotionDiffuse	0.491	0.681	0.782	0.630	3.113	9.410	1.55
	T2M-GPT	0.491	0.680	0.775	0.116	3.118	9.761	1.86
	MMM	0.504	0.696	0.794	0.080	2.998	9.411	1.16
	MoMask	0.521	0.713	0.807	0.045	2.958	9.620	1.24
	BAMM	0.525	0.720	0.814	0.055	2.919	9.717	1.69
	MoMask++ ⁱⁿ	0.528	0.718	0.811	0.072	2.912	–	1.23
	MoMask++ ^{cra}	0.517	0.709	0.803	0.069	2.948	–	1.19
Gen. & Und.	MotionGPT	0.492	0.681	0.778	0.232	3.096	9.528	2.01
	MotionGPT3	0.553	0.747	0.837	0.208	2.725	9.700	1.02
	UniMo	0.539	0.738	0.831	0.177	2.768	10.042	1.92
	MUGEN (Ours)	0.579	0.772	0.859	0.087	2.629	9.510	0.83

Table 1: Text-to-motion generation on HumanML3D. MUGEN has the best retrieval precision and matching distance in the table, both better than the real-motion reference, the best FID among language-model generators, and the Diversity closest to the real-motion reference (9.510 vs. 9.503). The masked-codebook pipelines keep the FID lead (0.045–0.080); the best of their R@1 values trails MUGEN by 0.051. Baseline numbers are quoted from their original papers; MoMask++ (Guo et al., 2025) appears as its in-context (ⁱⁿ) and cross-attention (^{cra}) conditioning variants.

Type	Method	R@1↑	R@2↑	R@3↑	FID↓	CLIP score↑	Diversity	MM
	Real motions	0.940±.001	0.976±.001	0.985±.001	0.001±.000	0.837±.000	19.5–19.8	–
Gen. only	MDM	0.503±.002	0.653±.002	0.727±.002	57.783±.092	0.481±.001	–	13.412±.231
	T2M-GPT	0.618±.002	0.773±.002	0.812±.002	32.629±.087	0.573±.001	–	9.172±.181
	StableMoFusion (Huang et al., 2024)	0.679±.002	0.823±.002	0.888±.002	27.801±.063	0.605±.001	–	9.064±.138
	MARDM	0.659±.002	0.812±.002	0.860±.002	26.878±.131	0.602±.001	–	9.812±.287
	MoMask	0.777±.002	0.888±.002	0.927±.002	17.404±.051	0.664±.001	–	8.183±.184
	MoMask++ ⁱⁿ	0.805±.002	0.904±.002	0.938±.001	15.56±.071	0.684±.001	–	6.556±.178
	MoMask++ ^{cra}	0.802±.001	0.905±.002	0.938±.001	15.06 ±.065	0.685±.001	–	7.259±.180
Gen. & Und. MUGEN (Ours)		0.815 ±.003	0.916 ±.002	0.947 ±.001	21.05±.074	0.698 ±.001	19.79±.037	6.09±.320

Table 2: Text-to-motion generation on SnapMoGen dataset. MUGEN is the one entry that also does understanding. It leads the multi-stage discrete state of the art at every retrieval rank and on CLIP score, and its Diversity is inside the ground-truth range. FID is the one column where the residual codebook pipelines stay ahead. Baseline numbers are quoted from the SnapMoGen benchmark report; superscripts as in Table 1.

Type	Method	R@1↑	R@2↑	R@3↑	MM-Dist↓	BLEU@1↑	BLEU@4↑	ROUGE↑	CIDEr↑	BERTScore↑
	Real motions	0.523	0.725	0.828	2.901	–	–	–	–	–
Und. only	LaMP-M2T (Li et al., 2025)	0.547	–	0.831	2.808	47.8	13.04	37.1	28.9	32.7
Gen. & Und.	TM2T	0.516	–	0.823	2.935	48.9	7.00	38.1	16.8	32.2
	MotionGPT	0.543	–	0.827	2.821	48.2	12.5	37.4	29.2	32.4
	MoTe (Wu et al., 2024)	0.577	–	0.871	2.649	46.7	11.15	37.4	31.5	30.3
	MotionGPT3	0.573	0.773	0.864	2.426	59.083	19.412	46.173	28.721	35.231
	UniMo	–	–	–	–	63.10	19.74	48.8	46.69	54.26
	MUGEN (Ours)	0.586	0.784	0.873	2.536	60.753	21.980	47.367	50.406	43.369

Table 3: Motion understanding on the HumanML3D dataset. Rows are grouped by what the reported model supports: the captioning-specific single-task model LaMP-M2T against models that also generate motion. MUGEN trains no captioning-specific variant, yet leads every reported retrieval rank together with BLEU@4 and CIDEr; UniMo, trained with curated chain-of-thought annotations, leads BLEU@1, ROUGE, and BERTScore and reports no retrieval columns.

Method	Params (M)	GFLOPs /motion	Latency (ms)	Throughput (motion/s)
MoMask++	150.2	105.5	55	55.1
MotionGPT3	299.7	95.5	136	53.2
MUGEN (Ours)	222.3	11.1	9	325.2

Table 4: Inference cost of generating one motion on HumanML3D. MUGEN spends $8.6\text{--}9.5\times$ less compute and responds $6\text{--}14\times$ faster than MoMask++ and MotionGPT3, with the highest batched throughput.

baseline stays ahead. SnapMoGen understanding has no published baseline and is reported in the

appendix.

4.4 Inference Cost

MUGEN reaches the quality above with K cached language-model steps, one structured Gaussian draw, and one decoder pass, a budget no baseline in Tables 1 and 2 matches. Table 4 quantifies the gap on a single NVIDIA L4, with each public checkpoint at the operating point of its reported results. The advantage is structural: MoMask++ runs 18 masked-decoding steps across four residual scales with two guided passes per step, MotionGPT3 emits

HumanML3D	R@1↑	R@2↑	R@3↑	FID↓	Div.	MM
<i>Latent budget K (depth-routed rollout)</i>						
$K=1$	0.574	0.771	0.859	0.109	9.50	0.67
$K=2$	0.579	0.772	0.859	0.087	9.51	0.83
$K=4$	0.577	0.774	0.862	0.091	9.46	0.70
$K=8$	0.567	0.763	0.853	0.111	9.45	0.91
$K=16$	0.566	0.761	0.852	0.143	9.46	0.74
<i>Rollout interface, at the $K=2$ budget</i>						
Last layer only	0.574	0.770	0.860	0.123	–	1.05

Table 5: Latent budget and rollout depth on the HumanML3D test split. Retrieval is flat over $K=1-4$ and falls beyond it, while FID is best at the small budgets. its motion latents one position at a time through an iterative diffusion head, and diffusion pipelines take tens to hundreds of denoising steps. Batching narrows the latency gap to about $6\times$ without closing it. The saving lies in inference steps rather than model size: MUGEN’s parameter count and peak memory both sit between the two baselines’. The measurement protocol and comparison scope are in the appendix.

4.5 Ablations

Table 5 varies the two structural choices of the rollout: the latent budget K , which sets the “codebook” size and, with key-value caching, the inference step count; and the depth at which the slots are read out.

A small budget suffices, on both benchmarks. Retrieval holds over $K=1-4$ and falls beyond it on HumanML3D; SnapMoGen agrees, with every retrieval metric and the CLIP score peaking at $K=4$ (table in the appendix). FID separates the two: best at the small budgets on HumanML3D, best at $K=16$ on SnapMoGen over a far narrower spread, a gain that costs 1.4 R@1 points. Since inference cost grows with K , we take the smallest budget that holds quality, $K=2^\dagger$ on HumanML3D and $K=4$ on SnapMoGen. Calibrated sampling beats μ -decode on FID at every budget on both datasets, at a cost of ≈ 0.6 R@1 points, and $\tau=1.0$ is never validation-optimal (the appendix reports the full sweep).

Routing the read-out over depth. Depth routing improves FID by 29% at fixed budget ($0.123 \rightarrow 0.087$), more than any move inside the $K=1-4$ region above, and the two slots take complementary roles (Fig. 3): k_0 keeps 98% of its routing mass in layers 9–11, while k_1 scans layers 5–11. They pick the same top layer on only 49% of test prompts, so the pair is not a redundant clone. The router adds a small cross-attention stack and MLP over per-layer states the backbone has already computed.

Training	FID↓	R@1↑	CIDEr↑	BERT↑	Params
Gen. only	0.107 $\pm$.004	0.578 $\pm$.002	–	–	221.9M
Und. only	–	–	52.8	44.8	222.3M
Joint, no floor	0.100 $\pm$.004	0.574 $\pm$.002	40.7	39.2	222.3M
Joint, $\lambda=0.25$	0.089 $\pm$.003	0.576 $\pm$.002	53.5	44.7	222.3M
Joint, $\lambda=1^\dagger$	0.087 $\pm$.004	0.579 $\pm$.002	50.4	43.4	222.3M
Joint, $\lambda=4$	0.111 $\pm$.003	0.571 $\pm$.002	49.7	43.2	222.3M

Table 6: Unified-training ablation on the HumanML3D test set; λ is the captioning weight λ_{m2t} , † the reported flagship, and BERT abbreviates BERTScore.

Unified training improves generation. Table 6 varies the training objective at the fixed flagship recipe. Dropping the captioning branch makes generation worse, not better (FID 0.107 vs. 0.087, disjoint confidence intervals, unchanged R@1): the understanding branch regularizes the shared motion representation. The reverse cost is small: the flagship stays within 5% of a dedicated captioner’s CIDEr while adding only a 0.39M projector (+0.18%), where task-specialized models would double the footprint. Removing the caption-loss floor degrades both directions at once.

5 Conclusion

We presented MUGEN, a unified motion–language framework that replaces discrete motion tokens with a small set of continuous latent slots shared across generation and understanding. Depth-routed latent extraction and a calibrated low-rank factor head make this interface effective at a decoding cost of K language-model steps, one draw, and one decoder pass, an order of magnitude below the measured inference cost of the strongest multi-stage baselines. MUGEN delivers strong and consistent results across retrieval, alignment, generation, and captioning benchmarks. These results show that a unified system does not need to buy retrieval, alignment, and captioning quality with codebook hierarchies or iterative decoding—FID is the one remaining gap to the masked-codebook family—and that a continuous latent interface is a simple and competitive foundation for motion–language modeling.

References

- Javed A Aslam and Emine Yilmaz. 2005. A geometric interpretation and analysis of r-precision. In *Proceedings of the 14th ACM international conference on Information and knowledge management*, pages 664–671.
- Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, et al. 2025. *SmolLM2: When smol goes big – data-centric*

- training of a small language model. *arXiv preprint arXiv:2502.02737*.
- Huiwen Chang, Han Zhang, Lu Jiang, Ce Liu, and William T. Freeman. 2022. [Maskgit: Masked generative image transformer](#). In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11305–11315.
- Xin Chen, Biao Jiang, Wen Liu, Zilong Huang, Bin Fu, Tao Chen, and Gang Yu. 2023. Executing your commands via motion diffusion in latent space. In *CVPR*, pages 18000–18010.
- Wenxun Dai, Ling-Hao Chen, Jingbo Wang, Jinpeng Liu, Bo Dai, and Yansong Tang. 2024. Motionlcm: Real-time controllable motion generation via latent consistency model. In *ECCV*, pages 390–408. Springer.
- Ke Fan, Shunlin Lu, Minyue Dai, Runyi Yu, Lixing Xiao, Zhiyang Dou, Junting Dong, Lizhuang Ma, and Jingbo Wang. 2025. Go to zero: Towards zero-shot motion generation with million-scale data. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 13336–13348.
- Chuan Guo, Inwoo Hwang, Jian Wang, and Bing Zhou. 2025. Snapmogen: Human motion generation from expressive texts. *arXiv preprint arXiv:2507.09122*.
- Chuan Guo, Yuxuan Mu, Muhammad Gohar Javed, Sen Wang, and Li Cheng. 2024. Momask: Generative masked modeling of 3d human motions. In *CVPR*, pages 1900–1910.
- Chuan Guo, Shihao Zou, Xinxin Zuo, Sen Wang, Wei Ji, Xingyu Li, and Li Cheng. 2022a. Generating diverse and natural 3d human motions from text. In *CVPR*, pages 5152–5161.
- Chuan Guo, Xinxin Zuo, Sen Wang, and Li Cheng. 2022b. [Tm2t: Stochastic and tokenized modeling for the reciprocal generation of 3d human motions and texts](#). In *ECCV*, page 580–597, Berlin, Heidelberg. Springer-Verlag.
- Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. 2017. Gans trained by a two time-scale update rule converge to a local nash equilibrium. *Advances in neural information processing systems*, 30.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising diffusion probabilistic models. In *Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS ’20*, Red Hook, NY, USA. Curran Associates Inc.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. [LoRA: Low-rank adaptation of large language models](#). In *International Conference on Learning Representations*.
- Yiheng Huang, Hui Yang, Chuanchen Luo, Yuxi Wang, Shibiao Xu, Zhaoxiang Zhang, Man Zhang, and Junran Peng. 2024. Stablemofusion: Towards robust and efficient diffusion-based motion generation framework. In *Proceedings of the 32nd ACM International Conference on Multimedia*, pages 224–232.
- Andrew Jaegle, Felix Gimeno, Andy Brock, Oriol Vinyals, Andrew Zisserman, and Joao Carreira. 2021. Perceiver: General perception with iterative attention. In *International conference on machine learning*, pages 4651–4664. PMLR.
- Eric Jang, Shixiang Gu, and Ben Poole. 2017. Categorical reparameterization with gumbel-softmax. In *International conference on learning representations*.
- Biao Jiang, Xin Chen, Wen Liu, Jingyi Yu, Gang Yu, and Tao Chen. 2023. Motiongpt: Human motion as a foreign language. *Advances in Neural Information Processing Systems*, 36:20067–20079.
- Doyup Lee, Chiheon Kim, Saehoon Kim, Minsu Cho, and Wook-Shin Han. 2022. [Autoregressive image generation using residual quantization](#). In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11513–11522.
- Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosior, Seungjin Choi, and Yee Whye Teh. 2019. Set transformer: A framework for attention-based permutation-invariant neural networks. In *International conference on machine learning*, pages 3744–3753. PMLR.
- Zhe Li, Weihao Yuan, Yisheng He, Lingteng Qiu, Shenhao Zhu, Xiaodong Gu, Weichao Shen, Yuan Dong, Zilong Dong, and Laurence Yang. 2025. Lamp: Language-motion pretraining for motion generation, retrieval, and captioning. In *International Conference on Learning Representations*, volume 2025, pages 84238–84250.
- Jing Lin, Ailing Zeng, Shunlin Lu, Yuanhao Cai, Ruimao Zhang, Haoqian Wang, and Lei Zhang. 2023. Motion-x: A large-scale 3d expressive whole-body human motion dataset. *Advances in Neural Information Processing Systems*, 36:25268–25280.
- Francesco Locatello, Dirk Weissenborn, Thomas Unterthiner, Aravindh Mahendran, Georg Heigold, Jakob Uszkoreit, Alexey Dosovitskiy, and Thomas Kipf. 2020. Object-centric learning with slot attention. In *Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS ’20*, Red Hook, NY, USA. Curran Associates Inc.
- Shunlin Lu, Jingbo Wang, Zeyu Lu, Ling-Hao Chen, Wenxun Dai, Junting Dong, Zhiyang Dou, Bo Dai, and Ruimao Zhang. 2025. [Scamo: Exploring the scaling law in autoregressive motion generation model](#). In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 27872–27882.
- Zichong Meng, Yiming Xie, Xiaogang Peng, Zeyu Han, and Huaizu Jiang. 2025. Rethinking diffusion for

- text-driven human motion generation: Redundant representations, evaluation, and masked autoregression. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 27859–27871.
- Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, pages 311–318.
- Maitreya Patel, Jingtao Li, Weiming Zhuang, Yezhou Yang, and Lingjuan Lyu. 2026. Vibetoken: Scaling 1d image tokenizers and autoregressive models for dynamic resolution generations. In *CVPR (CVPR)*.
- Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. 2018. [Deep contextualized word representations](#). In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 2227–2237, New Orleans, Louisiana. Association for Computational Linguistics.
- Mathis Petrovich, Michael J Black, and Gül Varol. 2022. Temos: Generating diverse human motions from textual descriptions. In *European conference on computer vision*, pages 480–497. Springer.
- Mathis Petrovich, Michael J Black, and Gül Varol. 2023. Tmr: Text-to-motion retrieval using contrastive 3d human motion synthesis. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 9488–9497.
- Ekkasit Pinyoanuntapong, Muhammad Usama Saleem, Pu Wang, Minwoo Lee, Srijan Das, and Chen Chen. 2024a. Bamm: bidirectional autoregressive motion model. In *European Conference on Computer Vision*, pages 172–190. Springer.
- Ekkasit Pinyoanuntapong, Pu Wang, Minwoo Lee, and Chen Chen. 2024b. Mmm: Generative masked motion model. In *CVPR*, pages 1546–1555.
- Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. [Learning transferable visual models from natural language supervision](#). In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 8748–8763. PMLR.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9.
- Anna Rogers, Olga Kovaleva, and Anna Rumshisky. 2020. [A primer in BERTology: What we know about how BERT works](#). *Transactions of the Association for Computational Linguistics*, 8:842–866.
- Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Bjorn Ommer. 2022. [High-Resolution Image Synthesis with Latent Diffusion Models](#). In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10674–10685, Los Alamitos, CA, USA. IEEE Computer Society.
- Kihyuk Sohn, Xinchun Yan, and Honglak Lee. 2015. Learning structured output representation using deep conditional generative models. In *Proceedings of the 29th International Conference on Neural Information Processing Systems - Volume 2, NIPS’15*, page 3483–3491, Cambridge, MA, USA. MIT Press.
- Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. 2023. Consistency models. In *Proceedings of the 40th International Conference on Machine Learning, ICML’23*. JMLR.org.
- Guy Tevet, Brian Gordon, Amir Hertz, Amit H Bermano, and Daniel Cohen-Or. 2022a. Motionclip: Exposing human motion generation to clip space. In *European Conference on Computer Vision*, pages 358–374. Springer.
- Guy Tevet, Sigal Raab, Brian Gordon, Yonatan Shafir, Daniel Cohen-Or, and Amit H Bermano. 2022b. Human motion diffusion model. *arXiv preprint arXiv:2209.14916*.
- Aaron Van Den Oord, Oriol Vinyals, et al. 2017. Neural discrete representation learning. *Advances in neural information processing systems*, 30.
- Ramakrishna Vedantam, C Lawrence Zitnick, and Devi Parikh. 2015. Cider: Consensus-based image description evaluation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4566–4575.
- Guocun Wang, Kenkun Liu, Jing Lin, Guorui Song, Jian Li, and Xiaoguang Han. 2026. [Unimo: unified motion generation and understanding with chain of thought](#). In *Proceedings of the Fortieth AAAI Conference on Artificial Intelligence and Thirty-Eighth Conference on Innovative Applications of Artificial Intelligence and Sixteenth Symposium on Educational Advances in Artificial Intelligence, AAAI’26/IAAI’26/EAAI’26*. AAAI Press.
- Bizhu Wu, Jinheng Xie, Keming Shen, Zhe Kong, Jianfeng Ren, Ruibin Bai, Rong Qu, and Linlin Shen. 2025. Mg-motionllm: A unified framework for motion comprehension and generation across multiple granularities. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 27849–27858.
- Yiming Wu, Wei Ji, Kecheng Zheng, Zicheng Wang, and Dong Xu. 2024. Mote: Learning motion-text diffusion model for multiple generation tasks. *arXiv preprint arXiv:2411.19786*.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, et al. 2025. [Qwen3 technical report](#). *arXiv preprint arXiv:2505.09388*.

Zhankai Ye, Bofan Li, Yukai Jin, Shuoqiu Li, Wei Wang, Yanfu Zhang, Shangqian Gao, and Xin Liu. 2026. Geomotiongpt: Geometry-aligned motion understanding with large language models. *arXiv preprint arXiv:2601.07632*.

Ye Yuan, Jiaming Song, Umar Iqbal, Arash Vahdat, and Jan Kautz. 2023. Physdiff: Physics-guided human motion diffusion model. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 16010–16021.

Jianrong Zhang, Yangsong Zhang, Xiaodong Cun, Yong Zhang, Hongwei Zhao, Hongtao Lu, Xi Shen, and Ying Shan. 2023a. Generating human motion from textual descriptions with discrete representations. In *CVPR*, pages 14730–14740.

Mingyuan Zhang, Zhongang Cai, Liang Pan, Fangzhou Hong, Xinying Guo, Lei Yang, and Ziwei Liu. 2024. Motiondiffuse: Text-driven human motion generation with diffusion model. *IEEE transactions on pattern analysis and machine intelligence*, 46(6):4115–4128.

Mingyuan Zhang, Xinying Guo, Liang Pan, Zhongang Cai, Fangzhou Hong, Huirong Li, Lei Yang, and Ziwei Liu. 2023b. Remodiffuse: Retrieval-augmented motion diffusion model. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 364–373.

Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q Weinberger, and Yoav Artzi. 2020. Bertscore: Evaluating text generation with bert. In *International Conference on Learning Representations*.

Chongyang Zhong, Lei Hu, Zihao Zhang, and Shihong Xia. 2023. **AttT2M: Text-Driven Human Motion Generation with Multi-Perspective Attention Mechanism**. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 509–519, Los Alamitos, CA, USA. IEEE Computer Society.

Bingfan Zhu, Biao Jiang, Sunyi Wang, Shixiang Tang, Tao Chen, Linjie Luo, Youyi Zheng, and Xin Chen. 2025. Motiongpt3: Human motion as a second modality. *arXiv preprint arXiv:2506.24086*.

A Experimental Setup Details

This appendix expands the experimental setup of the main paper.

Implementation. Stage one trains one ALAE per latent budget K (500 epochs, latent dimension $D=512$, four encoder and decoder layers). Stage two uses GPT-2 (Radford et al., 2019) as the language backbone ($L=12$ layers) and trains for 500 epochs with the decoupled anchor of the main paper; the factor rank is $r=64$ for every K and both datasets. The depth router uses two cross-attention blocks over the frozen ALAE latent queries and a shared two-layer MLP; its Gumbel temperature

anneals from 5 to 1.5 over the first 100 epochs, and evaluation always uses the deterministic tempered mixture. Generation and understanding are trained jointly on the same batches throughout. The ALAE encoder and decoder are frozen in stage two. All configurations of the K -scaling study are generated from a single base configuration with K as the only change.

Computing infrastructure. All experiments ran on a SLURM-managed academic GPU cluster under Linux. Each training run, for both stages and both datasets, used one NVIDIA B200 GPU, 16 CPU cores, and 160 GB of system memory; no experiment uses more than a single GPU. A stage-one run completes in under 6 GPU-hours; a full 500-epoch stage-two run takes about 6 GPU-hours on HumanML3D and up to about 21 GPU-hours on SnapMoGen at the largest latent budget. Evaluation jobs run in the same environment on a single B200 or L4 GPU. The software stack is Python 3.11, a PyTorch 2.11 nightly build with CUDA 12.8 (required by the B200 architecture), PyTorch Lightning 2.0, and Hugging Face Transformers 4.47 (4.51 for the backbone ablation of App. H.8, the minimum version that supports Qwen3); the released code pins the exact version of every dependency. The main paper compares the inference cost of the reported system against the two baselines on one shared GPU, and App. H.10 records the full measurement protocol.

Protocol and model selection. The sampling temperature is selected on the validation split only, and the test split is evaluated at the selected operating point under the official 20-replication protocol (means with 95% confidence intervals), with $\tau=1.0$ and μ -decode reported as references on HumanML3D (Table 11). Model selection likewise uses only the sampled validation split: the unified HumanML3D flagship takes the checkpoint with the best joint generation–understanding score of Eq. 1, so a single checkpoint serves both the generation and the understanding results reported for HumanML3D in the main paper, while the K -scaling study and the SnapMoGen flagship take the best-FID checkpoint. The temperature-calibration procedure itself is detailed in App. G.

The joint selection score. Every metric is first mapped to a higher-is-better 0–100 scale. Generation contributes four terms: retrieval $S_R=100(R@1+R@2+R@3)/3$; fidelity $S_F=100\tau_f/(\tau_f+FID)$ with $\tau_f=1$, a rational

map with no saturation regime; matching $S_M=100 \text{ MMDist}_{gt}/\text{MMDist}$, which rises above 100 when the model beats the real-motion reference (on SnapMoGen the evaluator’s matching score is a similarity, so the ratio is inverted); and diversity $S_D=100 \max(0, 1-|\text{Div}-\text{Div}_{gt}|/\text{Div}_{gt})$ with $\text{Div}_{gt}=9.5$ on HumanML3D, which penalizes deviation from the reference in either direction. They aggregate as the weighted geometric mean $\text{Avg}_G=(S_R^2 S_F^2 S_M S_D)^{1/6}$, so no term can compensate for another. Understanding contributes the plain benchmark average Avg_U of the motion-to-text retrieval mean 100 $(R@1+R@2+R@3)/3$ and $100\times\text{BLEU}@1$, $\text{BLEU}@4$, ROUGE , and CIDEr ; BERTScore joins this mean only where it is computed, and it is disabled during validation for cost, so selection uses the five-term mean. The selected checkpoint maximizes

$$S = \text{Avg}_G^\alpha \text{Avg}_U^{1-\alpha}, \quad \alpha = 0.5, \quad (1)$$

with every geometric-mean factor, in Avg_G and in Eq. 1, clamped below at 10^{-6} : the geometric mean ensures that degrading either direction degrades S , so the saved checkpoint must serve both.

B Evaluation Metrics: Definitions and Motivation

All metrics are computed in the feature space of each benchmark’s official evaluator: the contrastive text–motion evaluator of Guo et al. (2022a) on HumanML3D, and the TMR-style dual encoder of Guo et al. (2025) on SnapMoGen (Petrovich et al., 2023). Let $f(\cdot)$ and $g(\cdot)$ denote the evaluator’s motion and text encoders. Using each benchmark’s official evaluator keeps every number directly comparable with the published baselines.

FID (Heusel et al., 2017). Gaussians (μ_r, Σ_r) and (μ_g, Σ_g) are fitted to the evaluator features of the real and generated motions, and

$$\text{FID} = \|\mu_r - \mu_g\|_2^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}).$$

FID measures distributional fidelity: whether the generated set as a whole occupies the same region of feature space as real motion.

R-Precision (Aslam and Yilmaz, 2005). Each generated motion is scored against a pool containing its ground-truth description and randomly mismatched ones (pool size 32 on HumanML3D and 100 on SnapMoGen, following each benchmark), ranked by the Euclidean distance between $f(\hat{x})$ and $g(y)$. $R@k$ is the fraction of samples whose true de-

scription ranks in the top k ; it measures per-sample text–motion correspondence.

Matching distance (MM-Dist). The mean Euclidean distance $\|f(\hat{x}) - g(y)\|_2$ between each generated motion and its own description, a threshold-free companion to R-Precision.

CLIP score (SnapMoGen). The cosine similarity between the ℓ_2 -normalized motion and text embeddings of the benchmark’s CLIP-style dual encoder (Radford et al., 2021), averaged over the test set.

Diversity and MultiModality. Diversity is the mean pairwise feature distance over 300 random pairs of generated motions; it should match the real-motion reference, since values far below or above it indicate a mismatch with the true output spread. MultiModality is the mean pairwise distance among 30 motions generated for the same description, averaged over 100 descriptions; it measures the within-text variation that remains after conditioning.

Captioning metrics. $\text{BLEU}@n$ (Papineni et al., 2002) is modified n -gram precision with a brevity penalty; ROUGE is a recall-oriented measure based on the longest common subsequence; CIDEr (Vedantam et al., 2015) is a TF-IDF-weighted n -gram consensus score; BERTScore (Zhang et al., 2020) matches contextual embeddings and captures semantic similarity beyond exact n -gram overlap. The retrieval columns of the understanding table use the evaluator above with generated captions in place of the ground truth.

Why this set. Together these metrics separate the three requirements a text-to-motion system must satisfy at once: distributional fidelity (FID), per-sample text correspondence (R-Precision, MM-Dist, CLIP score), and variation (Diversity, MultiModality). The captioning side combines lexical evidence (BLEU, ROUGE, CIDEr) with semantic evidence (BERTScore). This is also the standard reporting set of both benchmarks, which keeps every comparison in the main paper on published ground.

C Complete Hyperparameters

Table 7 lists every final hyperparameter of the recipe that produces the reported results. SnapMoGen reaches the same effective batch size as HumanML3D through two-step gradient accumulation over batches of 128. All runs fix the global random seed to 1234 through Lightning’s

Hyperparameter	Value
<i>Stage one: ALAE</i>	
Optimizer	AdamW
Weight decay	10^{-4} (none on bias/norm)
Learning rate; schedule	3×10^{-4} ; cosine, 3% warmup
Batch size; epochs	512; 500
Slot dim D ; enc./dec. layers	512; 4 / 4
Heads; FFN; dropout	8; 2048; 0.05
Conv backbone	depth 3, 2 res blocks, dilation 3
$\lambda_{\text{rec}}/\lambda_{\text{joint}}/\lambda_{\text{perc}}/\lambda_{\text{orth}}$	1/0.5/10/1
<i>Stage two: language model</i>	
Backbone	GPT-2 base ($L=12$)
Frozen modules	ALAE enc./dec.; text embeddings
Optimizer	AdamW, $\beta=(0.9, 0.99)$, decay 0.01
Learning rate; schedule	10^{-4} ; 5-ep. warmup, cosine
Batch; epochs; precision	256 / 128×2 ; 500; fp32
Max sequence length	256
Router	2 cross-attn blocks, MLP 512
Router $s_s, s_d; \lambda_{\text{MI}}$	4, 4; 0.05
Gumbel temperature	$5 \rightarrow 1.5$, first 100 epochs
Factor rank r ; anchor λ_{lat}	64; 2
$\lambda_{\text{rec}}/\lambda_{\text{joint}}/\lambda_{\text{perc}}$	1/0.5/10
λ_{m2t} ; label smoothing	1; 0.1
M2T latent noise; loss floor	0.05; 2.8
M2T decoding	greedy, $\leq 64/128$ tokens
<i>Evaluation protocol</i>	
τ sweep (validation only)	$\tau \in \{0.5, 0.55, \dots, 1.0\}$
Test protocol	20 replications, batch 128
Random seed	1234 (all runs)

Table 7: Complete final hyperparameters. Slash-separated values are HumanML3D / SnapMoGen; every other value is shared across both datasets and all latent budgets K .

seed_everything, and the released configuration files reproduce every row unchanged.

D Router Implementation Details

This appendix expands the depth-routed weighted hidden states of the main paper.

Prompt memory. Let $\mathbf{h}_{1:N}^{(L)} \in \mathbb{R}^{N \times H}$ be the final-layer states over the N prompt positions. We project the entire sequence to the latent width and retain it as memory, $\mathbf{M}_{\text{txt}} = \mathbf{W}_{\text{mem}} \mathbf{h}_{1:N}^{(L)} \in \mathbb{R}^{N \times D}$, and supply the padding mask to the attention so that only real tokens are attended. Pooling the prompt into a single vector instead would leave the cross-attention with one key. Its softmax is then identically one, the block degenerates into a gated linear map of that vector, and the queries lose the ability to look up *which* words they should route on.

Latent queries and router head. The latent queries $\tilde{\mathbf{Q}} \in \mathbb{R}^{K \times D}$ are imported from the ALAE encoder and kept frozen, so slot k inherits the slot semantics the autoencoder already learned. They attend to the prompt memory through two cross-attention blocks of the same design as the ALAE encoder’s,

$$\mathbf{c} = \text{LN}(\text{CA}(\tilde{\mathbf{Q}}, \mathbf{M}_{\text{txt}})) \in \mathbb{R}^{K \times D},$$

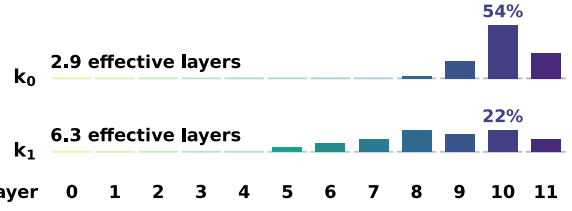


Figure 3: Layer-selection preference of the two slots on HumanML3D: k_0 is a narrow deep reader concentrated on layer 10, while k_1 is a wide scanner over layers 5–11.

and a shared two-layer MLP of hidden width 512 maps each context vector to the per-prompt correction $\Delta_k(y) \in \mathbb{R}^L$. Its output layer is zero-initialized. We set $s_s = s_d = 4$, which caps the logit margin at 8.

Temperature schedule. During training the Gumbel-Softmax temperature is annealed linearly from $\tau_r=5$ to 1.5 over the first 100 epochs, so early training explores soft mixtures over depth before the router commits. Evaluation uses the converged temperature rather than the annealed value: a detached inference pass would otherwise read the schedule at epoch zero and route almost uniformly.

Mutual-information objective. Let $\mathbf{p}_k = \text{softmax}(\mathbf{e}_k/\tau_r)$ be the noise-free routing distribution of slot k , let $\mathcal{H}[\cdot]$ denote entropy, and let $\bar{\cdot}$ denote the average over prompts in a batch. Per slot we maximize

$$\mathcal{I}_{\text{route}} = \mathcal{H}[\bar{\mathbf{p}}_k] - \overline{\mathcal{H}[\mathbf{p}_k]},$$

weighted by 0.05. The first term pushes different prompts to route differently, the second pushes each individual prompt to route decisively. Both are evaluated on $\mathbf{p}_k$ rather than on the sampled weights, since the injected noise would inflate the conditional entropy artificially. A plain entropy bonus is not a substitute: it rewards soft mixtures for every prompt instead of conditional switching.

Cost. The added cost over a standard rollout is the two-block cross-attention stack over the K latent queries plus the shared MLP. The per-layer states $\mathbf{H}$ are already produced by the backbone’s forward pass, and the routing logits depend only on the prompt, so they are computed once per generation.

Per-verb routing. The scanner’s depth also tracks verb semantics (Fig. 4), reading shallow for ballistic/contact verbs and deep for object manipulation—structure a collapsed static router cannot show.

E Factor Head Implementation Details

Parameterization. Each routed slot state $\tilde{\mathbf{h}}_k$ is layer-normalized and linearly projected to the per-

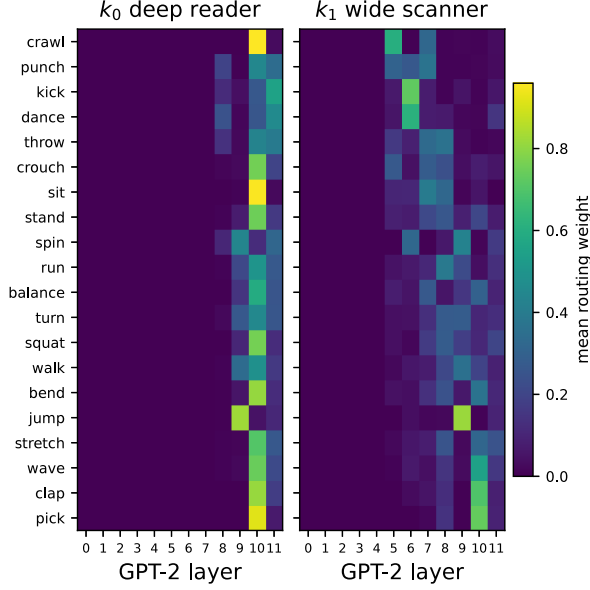


Figure 4: Per-verb layer preference on HumanML3D: k_0 (left) stays fixed on layer 10 for nearly every verb, whereas k_1 (right) shifts its preferred layer with the semantics of the action—shallow for ballistic/contact verbs (crawl, punch, kick), mid-depth for locomotion (run, walk, jump), and deep for object/hand actions (pick, clap, wave).

slot mean and diagonal scale, and the slot-pooled normalized states produce the factor amplitudes. The global basis $\mathbf{U} \in \mathbb{R}^{KD \times r}$ is initialized with unit-norm columns. The linear layer producing the log-amplitudes $\ell(y)$ has its bias initialized to -2 , so $\mathbf{a} \approx e^{-2}$ and training starts near the diagonal model, with the factor part earning its amplitude from the likelihood.

Closed-form likelihood. The negative log-likelihood of the low-rank-plus-diagonal family is computed in closed form. The Woodbury identity and the matrix determinant lemma reduce it to diagonal operations plus one $r \times r$ Cholesky factorization per sample, so the cost is linear in KD rather than cubic, and no $KD \times KD$ covariance is ever formed.

Decoupled anchor. Written out, the decoupled anchor $\mathcal{L}_{\text{lat}}$ of the joint training objective is

$$\begin{aligned} \mathcal{L}_{\text{lat}} = & -\log \mathcal{N}(\mathbf{z}^*; \boldsymbol{\mu}, \text{sg}[\text{diag}(\boldsymbol{\sigma}^2)]) \\ & -\log \mathcal{N}(\mathbf{z}^*; \text{sg}[\boldsymbol{\mu}], \\ & \quad \mathbf{U} \text{diag}(\mathbf{a})^2 \mathbf{U}^\top + \text{diag}(\boldsymbol{\sigma}^2)), \end{aligned}$$

where $\text{sg}[\cdot]$ denotes stop-gradient. The first term trains only the mean under a frozen diagonal precision, so the diagonal scale acts on the mean update purely as a stop-gradient preconditioner. The second fits the full covariance to the detached residual. The factor parameters $(\mathbf{U}, \mathbf{a})$ therefore never appear in

any term that carries gradient to $\boldsymbol{\mu}$, so the mean update is invariant to the covariance fit.

F Joint Training Details

Loss weights. The decoupled anchor is weighted by $\lambda_{\text{lat}}=2$ and the motion-to-text term by $\lambda_{\text{m2t}}=1$.

Motion understanding branch. The caption cross-entropy uses label smoothing 0.1. We perturb the input latents at train time with zero-mean Gaussian noise of standard deviation 0.05, and we floor the caption loss at 2.8, reflecting the gradient below that value so the understanding direction cannot keep sharpening at the expense of generation. The unified-training ablation of App. H.9 shows this floor in fact protects both directions. Captions are decoded greedily with at most 64 new tokens.

G Sampling Protocol and Temperature Calibration

At evaluation time a motion latent is drawn in a single shot as

$$\begin{aligned} \bar{\mathbf{z}} = & \boldsymbol{\mu} + \tau (\mathbf{U}(\mathbf{a} \odot \boldsymbol{\epsilon}_1) + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}_2), \\ & \boldsymbol{\epsilon}_1, \boldsymbol{\epsilon}_2 \sim \mathcal{N}(0, \mathbf{I}), \end{aligned}$$

where the temperature τ scales all zero-mean perturbations around the mean and $\tau=0$ recovers deterministic mean decoding (μ -decode).

Because the covariance is fitted by maximum likelihood on training residuals, it is mildly overdispersed on held-out text, so a post-hoc temperature slightly below one is a legitimate calibration rather than a trick. Our protocol is strict about this: τ is selected *only on the validation split* (a single sweep over $\tau \in [0.5, 1.0]$), and the test split is evaluated once at the selected operating point, with $\tau=1.0$ and μ -decode reported as references on HumanML3D.

All quantitative claims use this *honest-sampling protocol*: model selection is performed under sampled validation, every reported metric, including diversity and MultiModality, comes from the same sampler at the same operating point, and test numbers use the official 20-replication protocol of each benchmark.

H Additional Experimental Results

H.1 K-Scaling on SnapMoGen

Table 8 repeats the budget sweep of the main paper on SnapMoGen. The retrieval optimum transfers: a small budget is best on both benchmarks, and leaving it costs precision. The FID landscape does not, since the two benchmarks place their best and

K	τ^*	R@1 $\uparrow$	R@2 $\uparrow$	R@3 $\uparrow$	FID $\downarrow$	CLIP $\uparrow$	Div.	MM
1	0.6	0.799	0.904	0.938	22.28	0.691	19.74	6.90
4	0.6	0.815	0.916	0.947	21.05	0.698	19.79	6.09
8	0.6	0.804	0.905	0.939	21.99	0.686	19.86	6.41
16	0.6	0.801	0.904	0.939	20.76	0.686	19.86	7.28

Table 8: Latent budget K on the SnapMoGen test split, the companion of the HumanML3D block reported in the main paper (τ^* = validation-selected temperature, 20 replications, ground-truth Diversity 19.5–19.8). Every retrieval metric and the CLIP score peak at $K=4$, while FID is best at $K=16$ over a spread of only 1.5 points.

	$s=0$	$s=0.25$	$s=0.5$	$s=0.75$	$s=1$	shuffle
FID $\downarrow$	23.54	16.95	6.89	1.70	0.65	14.23
R@1 $\uparrow$	0.828	0.926	0.962	0.957	0.939	0.422

Table 9: Oracle residual decoding on SnapMoGen test: decode $\mathbf{z} = \boldsymbol{\mu} + s(\mathbf{z}^* - \boldsymbol{\mu})$. FID and R@1 improve *together* along the true residual direction; batch-shuffled residuals (right) collapse R@1, showing the residual is text-conditional. $s=1$ equals the stage-one reconstruction ceiling.

worst FID budgets at opposite ends of the sweep. The $K=16$ FID gain here costs 1.4 R@1 points, so it does not move the recommended operating point away from $K=4$.

H.2 Oracle Residual Diagnosis

To localize the remaining FID gap we decode $\mathbf{z} = \boldsymbol{\mu} + s(\mathbf{z}^* - \boldsymbol{\mu})$, replacing the sampler with a scaled *true* residual at zero training cost (Table 9). Three facts follow. (i) The representation is not the bottleneck: with only half the true residual ($s=0.5$) FID drops to 6.89, far below the 20.8–22.3 sampled FIDs of Table 8—the representation retains substantial oracle headroom. The continuous slot representation, in other words, already contains the motions the benchmark asks for. (ii) FID and R-Precision rise *together* along the true residual direction, suggesting that the FID–precision trade-off observed under isotropic or diagonal sampling families is an artifact of the sampling family, not a property of the task. (iii) Batch-shuffling the residuals keeps FID moderate but collapses R@1 to 0.42: the missing variance is text-conditional and directionally structured. Together these pin the residual FID gap on the conditional sampler—the single draw that single-shot generation allows—motivating the structured factor head, and explaining why naïve noise injection cannot close it.

H.3 Latent Distribution Family

Distribution family. Table 10 traces the path to the factor head on SnapMoGen. The KL-regularized diagonal head cannot be rescued by test-time tem-

Latent head (SnapMoGen $K=4$, val)	FID $\downarrow$	R@1 $\uparrow$
KL-pinned diagonal (σ pinned at 1)	25.8 †	–
Diagonal NLL	33.7	–
Cross-slot AR chain (sched. sampling)	29.2	–
Factor head, joint ML (no decoupling)	47.5 ‡	0.13 ‡
Factor head, decoupled (ours)	22.97	0.775

Table 10: Distribution-family ablation on SnapMoGen.

† Best value across a $\tau \in [0, 3]$ sweep—the KL-pinned model’s temperature response is nearly flat because training deletes text-conditional variance. ‡ Joint ML collapses via variance-eats-gradient (shown at epoch 9, when the run was stopped).

Decoding	R@1 $\uparrow$	R@2 $\uparrow$	R@3 $\uparrow$	FID $\downarrow$	MM-Dist $\downarrow$	MM
μ -decode	0.585	0.778	0.863	0.131	2.611	–
$\tau=1.0$	0.555	0.751	0.844	0.151	2.735	1.35
$\tau^*=0.6$	0.579	0.772	0.859	0.087	2.629	0.83

Table 11: Sampling-protocol ablation on the HumanML3D test set: deterministic μ -decode versus uncalibrated ($\tau=1.0$) and validation-calibrated ($\tau^*=0.6$) sampling. Calibrated sampling trades ≈ 0.6 R@1 points for a 34% lower FID than μ -decode.

perature (a $\tau \in [0, 3]$ sweep moves FID by <3 points): with the variance pinned, training removes within-text variance from the mean itself. Replacing KL with a diagonal NLL calibrates $\sigma(y)$ but plateaus hard at FID 33.7—the diagonal independence assumption is the binding constraint, consistent with the measured cross-slot correlations ($|\rho| \approx 0.34\text{--}0.52$). An autoregressive cross-slot chain restores correlation but is brittle—and it surrenders the single-shot budget, replacing the single draw with sequential sampling that peaks at 29.2 (epoch 49) and then overfits its own conditional mechanism. The single-shot factor head reaches 22.97, with *rising* R@1 indicating sampling in the correct residual directions (App. H.2).

Decoupled optimization is necessary. Under naively joint maximum likelihood the factor amplitudes absorb the mean error within a few epochs ($\bar{a}$: 0.15 $\rightarrow$ 4.2) while retrieval collapses to near-random—the variance-eats-gradient failure. The decoupled anchor removes the failure without clamps: the mean gradient is bit-identical regardless of the covariance parameters, and the amplitude trajectory becomes rise-then-fall, tracking the shrinking residual.

H.4 Sampling versus μ -decode

Table 11 isolates the sampling protocol on HumanML3D. The single calibrated draw improves FID by 34% over deterministic μ -decode (0.131 $\rightarrow$ 0.087) at a small retrieval cost (0.585 $\rightarrow$ 0.579

SnapMoGen test	MUGEN (Ours)	Real captions
R@1↑	0.600	0.945
R@2↑	0.750	0.980
R@3↑	0.819	0.988
CLIP score↑	0.580	0.840
BLEU@1↑	64.655	—
BLEU@4↑	23.492	—
ROUGE↑	37.675	—
CIDEr↑	9.392	—
BERTScore↑	26.354	—

Table 12: Motion understanding on the SnapMoGen test set, from the same flagship checkpoint and the same operating point as the SnapMoGen generation results of the main paper, under the official 20-replication protocol. The right column is the real-caption reference measured in the same run, which upper-bounds the retrieval columns. All 95% confidence intervals are below ± 0.002 on the retrieval columns and below ± 0.05 on the captioning columns. We are not aware of published captioning results on SnapMoGen, so no baseline row is available.

R@1), and the temperature exposes an explicit diversity–fidelity dial (MM 0.83 $\rightarrow$ 1.35 from $\tau=0.6$ to 1.0). On SnapMoGen the FID dividend is smaller and likewise costs ≈ 0.01 R@1—consistent with the smaller dataset leaving a more over-dispersed ML covariance. In both cases $\tau=1.0$ is never optimal, confirming that validation-side calibration is a necessary component of an honest-sampling protocol rather than an optional refinement.

Across latent budgets on both benchmarks, the HumanML3D μ -decode $\rightarrow$ best- τ FID dividend is 0.026–0.030 for $K \leq 4$ and 0.064–0.078 for $K \geq 8$. On SnapMoGen the validation-selected temperature is 0.6 at every budget under the protocol’s R@1 selection rule, and best-temperature sampling beats μ -decode by 2.0–5.9 validation-FID points (26.4 $\rightarrow$ 23.3 at $K=4$).

H.5 Motion Understanding on SnapMoGen

Table 12 reports the understanding direction on SnapMoGen. The captioning branch is trained jointly with generation and reads the same frozen latent slots, so these numbers come from the SnapMoGen generation checkpoint of the main paper at no additional training cost. One protocol difference from HumanML3D is worth stating: the SnapMoGen checkpoint is selected by FID alone (App. A), so unlike the HumanML3D flagship it receives no understanding-side model selection.

Retrieval. Generated captions retrieve their own motion at R@1 0.600, well below the 0.945 of the human-written captions. This is the opposite of

the HumanML3D picture, where the same branch retrieves *above* the real-caption reference at every rank, and it mirrors the generation side: SnapMoGen is the harder benchmark in both directions for a single shared checkpoint.

Why the captioning scores are not comparable across benchmarks. A SnapMoGen clip carries six reference captions averaging 49 words with body-part-level detail, against three references averaging 13 words on HumanML3D. Our SnapMoGen captions match that length (median 49 words), so the gap is not a brevity artifact. Under those long, highly specific references n -gram precision stays high, in fact above the HumanML3D values reported in the main paper (BLEU@1 64.7 vs. 60.8, BLEU@4 23.5 vs. 22.0), while the consensus-weighted and embedding-based scores fall sharply (CIDEr 9.4 vs. 50.4, BERTScore 26.4 vs. 43.4). The two families disagree because they weight different things: the model produces fluent, correctly scaled descriptions of the coarse motion, and it misses the rare, high-information detail that carries the TF-IDF weight in CIDEr. Captioning scores should therefore be read within a benchmark, never across the two.

H.6 Motion Generation Case Study

Table 13 makes the aggregate HumanML3D generation numbers of the main paper concrete on individual motions, and shows that the two baselines fail in different directions. MotionGPT3 converts stationary prompts into locomotion: it travels further than the reference on all five clips, most starkly on “*stands still*”, where the render shows a full walking sequence. MoMask++ follows trajectories more faithfully—measured over the whole test set its trajectory error is statistically indistinguishable from ours—but its poses carry visible high-frequency noise, consistent with a jitter of $1.64\times$ the real level; MotionGPT3 sits at $0.61\times$ and is correspondingly over-smoothed. MUGEN is the only one of the three that stays close to the reference on both quantities at once. We emphasise that lower jitter is not by itself better: MotionGPT3 attains the lowest absolute jitter of the three while being the furthest from real motion on that axis, so the quantity to read is the distance to the reference rather than the raw value.

H.7 Motion Understanding Case Study

Table 14 illustrates the understanding gap behind the aggregate HumanML3D metrics of the main paper. The failures on the MotionGPT3 row are not paraphrase noise: the model replaces the action

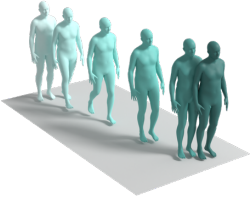
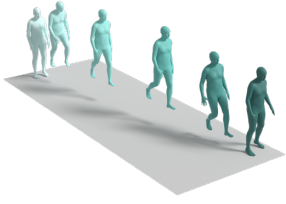
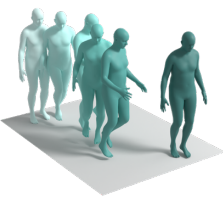
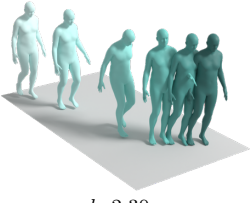
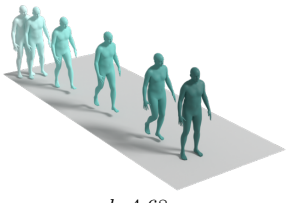
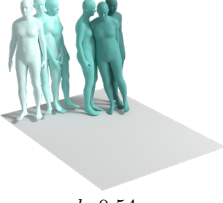
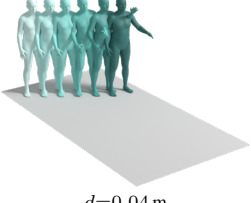
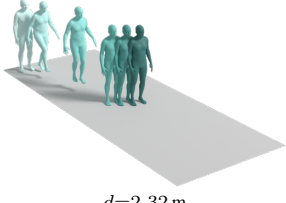
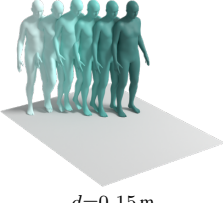
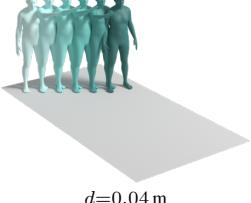
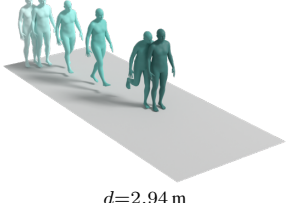
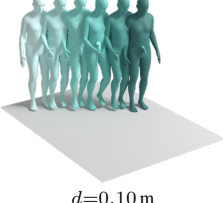
Prompt	<i>“person stands still and aggressively points straight forward”</i>	<i>“a person who is standing on a balance beam takes three steps forward and then steps down off the beam.”</i>	<i>“a person starts a jogging on the place”</i>
MotionGPT3	 $d=2.55$ m	 $d=5.10$ m	 $d=1.59$ m
MoMask++	 $d=2.30$ m	 $d=4.68$ m	 $d=0.54$ m
MUGEN (Ours)	 $d=0.04$ m	 $d=2.32$ m	 $d=0.15$ m
Real	 $d=0.04$ m	 $d=2.94$ m	 $d=0.10$ m

Table 13: Motion generation case study on HumanML3D test. All three systems receive the *same* prompt and the *same* target length as the reference motion; poses are rendered light to dark over time, and the four rows of a column share one camera and floor tile, so sizes may be compared vertically. The number under each render is the root horizontal displacement d in metres. Each prompt constrains how far the body should travel (teal), and both baselines overshoot that constraint on every clip shown: on “stands still” they cover 2.30 and 2.55 m where the reference covers 0.04 m, and on “jogging on the place” MotionGPT3 drifts 1.59 m against 0.10 m. Averaged over the five clips, the deviation $|d - d_{\text{real}}|$ is 0.19 m for MUGEN, 1.32 m for MoMask++ and 2.09 m for MotionGPT3. The clips are drawn from the 128 test motions that are non-inferior to *both* baselines on every physical axis we measure, a pool fixed before any figure was chosen; the per-clip displacement gap is therefore illustrative of that regime and is not an aggregate claim.

with a different one, drops the object that defines it, or loses the trajectory, while low-order n -grams (“a person . . .”) still match. This is the microscopic account of why the margin between the two systems widens with metric order, from +1.7 BLEU@1 to +2.6 BLEU@4 to +21.7 CIDEr in the aggregate metrics.

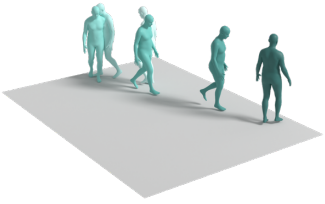
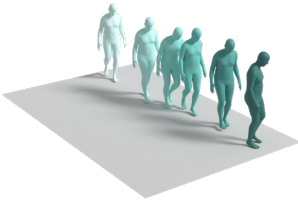
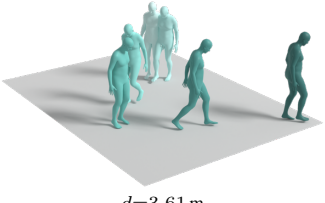
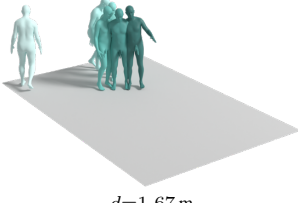
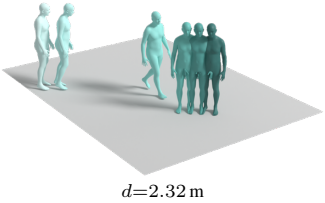
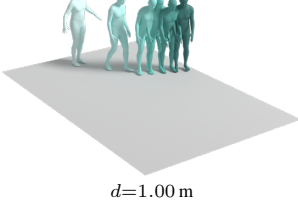
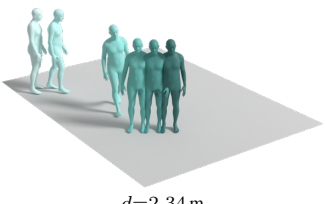
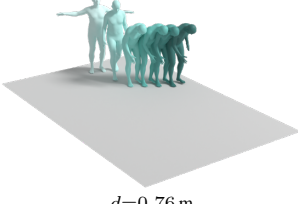
Prompt	<i>“a person walks forward and turns to the right.”</i>	<i>“a person walks up to a table and starts washing it.”</i>
MotionGPT3	 $d=3.82\text{ m}$	 $d=3.56\text{ m}$
MoMask++	 $d=3.61\text{ m}$	 $d=1.67\text{ m}$
MUGEN (Ours)	 $d=2.32\text{ m}$	 $d=1.00\text{ m}$
Real	 $d=2.34\text{ m}$	 $d=0.76\text{ m}$

Table 13: Motion generation case study on HumanML3D test (continued). On “walks forward and turns to the right” MotionGPT3 not only travels 3.82 m against the reference 2.34 m but also turns through 277° where the reference turns 91° and MUGEN through 104° , so its final pose faces away from the camera.

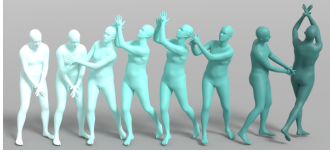
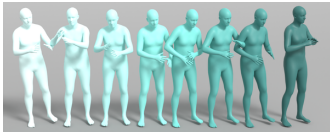
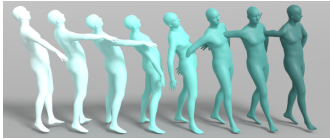
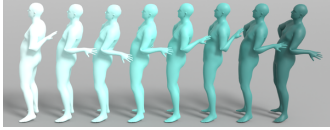
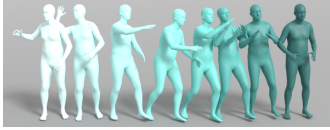
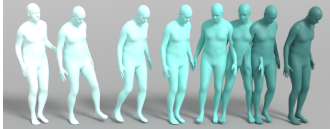
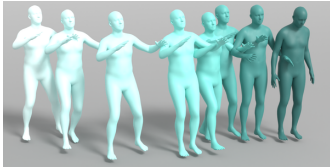
Input Motions			
Real	<i>“the person swings a golf club”</i>	<i>“a person sweeping the floor”</i>	<i>“a person underhand throws something with their right hand.”</i>
MotionGPT3	<i>“a man holds both hands together near his groin and then simultaneously lifts them up in front of him.”</i>	<i>“a person uses a hammer to swing an object, while holding it in its hands.”</i>	<i>“a person reaches right hand up and swings it in front of them”</i>
MUGEN (Ours)	<i>“a person swings a golf club.”</i>	<i>“a person is sweeping the floor.”</i>	<i>“a person throws something with their right hand.”</i>
Input Motions			
Real	<i>“the person is acting like a human chicken.”</i>	<i>“the person is lifting a dumbbell with the left hand.”</i>	<i>“a person throws something with their right hand.”</i>
MotionGPT3	<i>“a person bends over and rubs their hands together.”</i>	<i>“a person bends their arm at the elbow repeatedly”</i>	<i>“a person throws something, then catches it with both hands.”</i>
MUGEN (Ours)	<i>“a person is acting like a chicken.”</i>	<i>“a person is lifting a dumbbell with their left hand.”</i>	<i>“a person throws something with their right hand.”</i>
Input Motions			
Real	<i>“person is walking normally in a circle”</i>	<i>“a person side steps to the left and then to the right.”</i>	<i>“the person is dancing the waltz.”</i>
MotionGPT3	<i>“a man walks forward in an oval pattern”</i>	<i>“a person slightly moved forward and return”</i>	<i>“a person is dancing with a box step.”</i>
MUGEN (Ours)	<i>“a person walks in a circle”</i>	<i>“a person steps to the left, then steps back to the right.”</i>	<i>“a person is dancing the waltz.”</i>

Table 14: Motion understanding case study on HumanML3D test. Each block shows the input motion (rendered light to dark over time), a reference caption, and both models’ captions; phrases where MUGEN agrees with the reference are marked in teal. The MotionGPT3 captions admit no such alignment: the action, the object, or the trajectory is replaced or lost.

H.8 Language Backbone Ablation

Setup. This ablation varies the stage-two language backbone and nothing else. All five runs use the $K=2$ HumanML3D flagship recipe of the main paper: the same frozen stage-one ALAE checkpoint, factor rank $r=64$, the depth router, the joint captioning branch, 500 epochs, effective batch 256, and backbone learning rate 10^{-4} . Each configuration is derived from the flagship configuration file by changing only the backbone path (and, for the non-GPT-2 runs, the precision and the adapter block), so the five columns are key-for-key identical elsewhere. The router reads the backbone depth from the model configuration, which gives $L=12$ layers for GPT-2, $L=28$ for both Qwen3 models, and $L=24$ for SmolLM2.

The backbones are GPT-2 124M (Radford et al., 2019), Qwen3-0.6B and Qwen3-1.7B (Yang et al., 2025), and SmolLM2-1.7B (Ben Allal et al., 2025). The two Qwen3 sizes isolate model scale within a single family and tokenizer, while SmolLM2 matches Qwen3-1.7B in parameter count but comes from an independent pretraining pipeline. The billion-scale backbones are adapted with LoRA (Hu et al., 2022) ($r=16$, $\alpha=32$, dropout 0.05) on the seven projection matrices of every block; the motion-token embedding rows added to the vocabulary stay fully trainable and are excluded from weight decay, as in the flagship. Two departures from the flagship are forced by memory: these runs use bf16 mixed precision, because a 500-epoch fp32 run at this scale would take several days, and a micro-batch of 128 with two gradient-accumulation steps, because a micro-batch of 256 exceeds the memory of a single B200 at 1.7B parameters. Mixed precision is itself a confound, so we add a bf16 GPT-2 twin as a precision-matched control, and every backbone claim below is read against that twin rather than against the fp32 flagship. These runs use Transformers 4.51, the minimum version that supports Qwen3. Model selection, temperature calibration and the test protocol follow App. A: an eight-point validation temperature sweep, τ^* at maximal validation R@1, and the official 20-replication test protocol at τ^* . Stage-two wall-clock on one B200 was 5.3 h (GPT-2 bf16), 18.1 h (Qwen3-0.6B), 18.8 h (SmolLM2-1.7B) and 26.1 h (Qwen3-1.7B).

The backbone matters, but not through its size. At equal parameter count the two 1.7B backbones are far apart: SmolLM2 is 61% worse in FID than

Backbone	FID↓	R@1↑	B@4↑	CIDEr↑	Joint↑
GPT-2 124M [†]	0.087±.004	0.579±.002	22.0	50.4	66.8
GPT-2 124M [‡]	0.125±.005	0.575±.003	22.5	50.1	66.3
Qwen3-0.6B	0.080 ±.002	0.579±.003	20.0	46.8	65.3
Qwen3-1.7B	0.091±.003	0.585 ±.002	21.3	48.3	66.4
SmolLM2-1.7B	0.147±.004	0.564±.003	20.3	48.0	64.2

Table 15: Language backbone ablation on the HumanML3D test set, all rows at the best-joint checkpoint and its calibrated τ^* . [†]fp32, the flagship backbone; [‡]bf16, the precision-matched control. Every row except the flagship uses bf16 mixed precision, so the three billion-scale backbones should be read against that control rather than against the flagship. FID and R@1 are means with 95% confidence intervals over the official 20 replications; captioning is deterministic greedy decoding evaluated once, so B@4, CIDEr and the joint generation–understanding score carry no interval, and B@4 and CIDEr follow the $\times 100$ scale of the main paper.

Qwen3 (0.147 vs. 0.091, disjoint intervals), 0.021 lower in R@1, and 2.1 points lower on the joint score. Ordering Table 15 by parameter count instead gives 0.125 (124M), 0.080 (0.6B), 0.091 (1.7B) and 0.147 (1.7B), which is not monotone. What a backbone contributes is therefore a property of the pretrained model as a whole rather than of its capacity. The two 1.7B backbones also differ in tokenizer, vocabulary size (49k vs. 152k) and depth (24 vs. 28 layers), so we read the gap as a model-level effect and do not attribute it to pretraining data alone.

Checkpoint selection moves FID as much as the backbone does. Selecting by best validation FID instead of the flagship’s joint criterion leaves Qwen3-1.7B and SmolLM2 unchanged, since for both the two criteria pick the same epoch, but it moves Qwen3-0.6B from 0.080 (epoch 269) to 0.117 (epoch 329), a spread of 0.036 inside a single run. That spread is as large as the between-backbone differences at the top of the table, and every configuration here is a single seed, so we do not rank backbones by FID gaps of a few thousandths and we do not promote the nominally best FID of the table into a headline number. The joint score is the more stable readout, because it constrains both directions at once and is insensitive to noise in either one alone.

Captioning does not improve with a larger backbone. No swap helps the understanding direction: CIDEr is highest for the two GPT-2 runs (50.5 and 50.1) and lower for every billion-scale backbone (46.8 to 48.3), and BERTScore F1 is likewise highest for the fp32 GPT-2 run (43.7), with every other

backbone between 40.2 and 43.2. The captioning branch reads frozen ALAE latents through a projector, so its bottleneck is the motion representation rather than language-model capacity, which is consistent with the latent-side ablations of App. H.3.

Why the flagship keeps GPT-2. Qwen3-1.7B is the strongest alternative on the generation side, with the best R@1 in the table and an FID 27% below its precision-matched twin, but that advantage does not carry over to the joint objective the flagship is selected on, where GPT-2 remains highest. Combined with the captioning ordering above, the $14\times$ parameter cost, and a stage-two run that is five times longer, we keep GPT-2 as the backbone of the reported system and report this study as evidence that the pipeline is not bottlenecked by language-model capacity.

H.9 Unified-Training Ablation

Setup. This ablation varies the training objective and nothing else. All runs use the $K=2$ HumanML3D flagship recipe (App. H.8, first row) and are derived from the flagship configuration file by changing only the keys named below, so the rows of the unified-training table of the main paper are key-for-key identical elsewhere. *Generation only* disables the captioning task, so the motion-to-text branch is never built and the backbone is trained by the generation objective alone. *Understanding only* zeroes every generation-side loss term, so the backbone is trained by caption cross-entropy alone; the caption-loss floor of App. F is removed as well, since there is no generation left to protect. *Joint, no floor* keeps both objectives at $\lambda_{m2t}=1$ and removes only the floor. The last two rows move λ_{m2t} to 0.25 and 4 with the floor kept. Model selection uses the only criterion available to each regime: best validation FID for generation only, the understanding side of the joint score for understanding only, and the flagship’s joint criterion for the joint rows. Temperature calibration and the test protocol follow App. A; every swept row calibrates to $\tau^*=0.6$, and the understanding-only row needs no calibration because captions are decoded greedily. Each configuration is a single seed trained for the full 500 epochs on one B200 (3.7 h for generation only, 5.2 to 5.6 h otherwise), so as in App. H.8 we do not read FID gaps of a few thousandths. In that table, FID and R@1 are means with 95% confidence intervals over the official 20 replications. CIDEr and BERTScore F1 carry no interval: every row uses deterministic greedy decoding evaluated

once, except the flagship row, which quotes the 20-replication harvest of the main understanding table. The Params column counts all trained parameters of each checkpoint, excluding evaluation-only machinery.

The generation gain is not a selection artifact.

The generation-only row is selected by best validation FID, the flagship by the joint criterion of App. A. Selecting the flagship run by validation FID as well, the same criterion the generation-only row uses, still yields 0.090, so the FID gap read in the main paper (0.107 vs. 0.087) survives criterion-matched selection.

A dedicated captioner is barely better and does not survive the schedule.

The understanding-only run reaches CIDEr 52.8 and BERTScore F1 44.8, but its validation score improves for the last time at epoch 29 and never again over the remaining 470 epochs: with no floor and no competing objective the caption loss keeps falling while held-out quality stalls, the signature of memorization. The flagship stays within 5% of it in CIDEr (50.4) while being selected at epoch 239, and at $\lambda_{m2t}=0.25$ the joint model reaches 53.5, nominally above the dedicated model; given single-seed selection noise we read this as parity. Sharing the backbone thus costs the understanding direction little to nothing, and removes the need for the early stopping the dedicated model depends on.

The floor protects both directions. The floor was introduced to keep the understanding branch from sharpening at the expense of generation (App. F), but the naive-joint row shows its effect is not a one-sided trade: removing it degrades generation (FID 0.100, disjoint from 0.087) and degrades understanding even more (CIDEr 40.7, a 19% drop; BERTScore 39.2). An unconstrained caption term drags the shared backbone toward caption memorization and thereby hurts its own held-out captioning as well. The floor is best understood as a regularizer for both directions rather than as protection for one.

The operating point is insensitive to λ_{m2t} over a decade.

$\lambda_{m2t}=0.25$ and 1 are statistically indistinguishable on generation (0.089 vs. 0.087, overlapping intervals) and comparable on understanding, while $\lambda_{m2t}=4$ degrades generation (0.111, disjoint from 0.087) without helping understanding (49.7). The useful range is therefore wide below the default, and the upper bound lies between 1 and 4.

H.10 Computational Requirements

Protocol. The inference-cost table of the main paper reports what it costs to generate one motion with each of the three systems whose HumanML3D generations are compared in App. H.6. All three run on one NVIDIA L4 GPU, from their public checkpoints, over the same pinned list of captions and target lengths, and each at the operating point of its own reported results: MoMask++ at conditioning scale 5 with 18 masked-decoding steps, MotionGPT3 at guidance scale 7.5, and MUGEN at $\tau^*=0.6$ without classifier-free guidance. Each row is therefore the cost of reaching that row’s published quality, not a cost at some artificially equalized setting. Timing covers text to motion features on the GPU and excludes forward kinematics, denormalization and file I/O, which all three share. Latency is the median over 64 single-sample generations after warm-up, and throughput uses batches of 32. Parameter counts include the frozen text encoder each system loads at inference, a T5-v1.1-base encoder (109.6M) for MoMask++ and the language backbone itself for the other two. Target lengths are capped at 196 frames for every system because MoMask++ has a fixed positional grid at that length.

Discussion. The compute column and the latency column of that table do not rank the two baselines the same way. MotionGPT3 needs $8.6\times$ our FLOPs but $14.3\times$ our latency, so its serial decoding is latency-bound rather than compute-bound, while MoMask++ needs $9.5\times$ our FLOPs and only $5.8\times$ our latency because its masked steps are large dense products that keep the device busy. At batch 32 the per-motion factors are 5.9 and 6.1.

Scope of the claim. The saving is in inference steps, not in model size. At 222.3M parameters MUGEN is larger than MoMask++ (150.2M) and smaller than MotionGPT3 (299.7M), and its peak memory at batch 32 (1.66 GB) likewise sits between the two (0.73 and 3.50 GB), because decoding a clip in one pass materializes the whole feature tensor. The measurement is also specific to HumanML3D: MoMask++ spends the same 105.5 GFLOPs on every sample since its token grid and text padding are fixed, while our cost varies with caption length (9.9 to 12.1 GFLOPs across the sampled clips), so the ratios would move on a benchmark with longer motions. Training cost is likewise outside this table. All three rows evaluate public checkpoints and only ours was trained by us, so baseline training budgets are stated as published configurations rather

than measured GPU-hours: MoMask++ trains a residual VQ for 1000 epochs at batch 256 and a mask transformer for 500 epochs at batch 64, and MotionGPT3 uses a three-stage schedule. Our own budget is given in App. A.

H.11 What Did Not Help

A set-level mixture extension of the factor head ($M=8$ tied-covariance components) trained stably, but its mixture component was a consistent *net negative* on test FID at every temperature we evaluated (e.g. 22.07 vs. 21.18 at the same τ with the mixture disabled). The unimodal factor sampler remains our best configuration; realizing the oracle headroom in Table 9 remains open.